

Multi-Agent vs. Single-Agent LLM Code Review

A cross-model, grounding-verified evaluation of automated pull-request review quality

Lovepreet Singh

Kochava · lsingh@kochava.com

ABSTRACT

Automated code review by large language models (LLMs) is increasingly deployed in CI pipelines, yet rigorous, bias-controlled comparisons between review architectures remain scarce. We evaluate a multi-agent reviewer (*skwad*, built on claude-sonnet-4-6) against a single-agent baseline (GitHub's Claude CI reviewer, claude-sonnet-4-5-20250929) on **N=8** real pull requests from three production repositories spanning three difficulty tiers. Reviews were scored on a pre-registered seven-criterion rubric (0–3 each, 21-point maximum) by an *out-of-family* judge (gpt-5.4), eliminating same-model self-preference; each PR was judged three times under counterbalanced A/B blinding and combined by median vote. Crucially, the judge **grounded every claim against the repository** via file-read and grep tool calls, labelling each verified, contradicted, or unverifiable. *skwad* won all 8 head-to-head comparisons (Wilcoxon **p=0.0078**; Cliff's **d=0.78**, 95% BCa CI [0.50, 1.00] — large), with claims verified at **70.9%** versus 60.2% for the baseline across 643 adjudicated claims. A Krippendorff's- α reliability gate flagged the three most subjective criteria as low-consistency — a transparency caveat that also gates the harness's self-validity — while the headline statistics run on the full seven-criterion, 21-point total. We report a decisive pilot and recommend a 30-PR confirmatory study before full deployment.

KEYWORDS — LLM-as-judge · automated code review · multi-agent systems · inter-rater reliability · effect size · grounding verification

1 Introduction

Pull-request (PR) review gates merge velocity while consuming scarce senior attention. LLM reviewers promise to absorb the routine load, and tools such as GitHub's Claude CI reviewer now post review comments automatically. This raises an architectural question: does decomposing review into a coordinated team of specialized agents — each focused on correctness, security, testing, performance — produce materially better reviews than a single capable model prompted once?

We compare *skwad*, a multi-agent orchestrator that spawns specialized reviewer agents and synthesizes their findings, against the single-agent Claude CI reviewer. The central hazard in any such study is the evaluator: LLM

judges exhibit *self-preference*, scoring their own family's output more favorably [1], and reward superficial properties such as length and structure independently of substance [2]. A multi-agent system that merely emits more text could therefore *appear* superior without being so. Our design neutralizes both hazards, and adds per-claim repository grounding so that "quality" is measured as verifiable correctness, not prose.

Contributions: (i) a bias-controlled protocol pairing an out-of-family judge with per-claim grounding; (ii) a reliability gate that flags criteria the judge cannot score consistently and gates the harness's self-validity, rather than silently dropping them from the headline; and (iii) an honest accounting of what an N=8 pilot can and cannot establish.

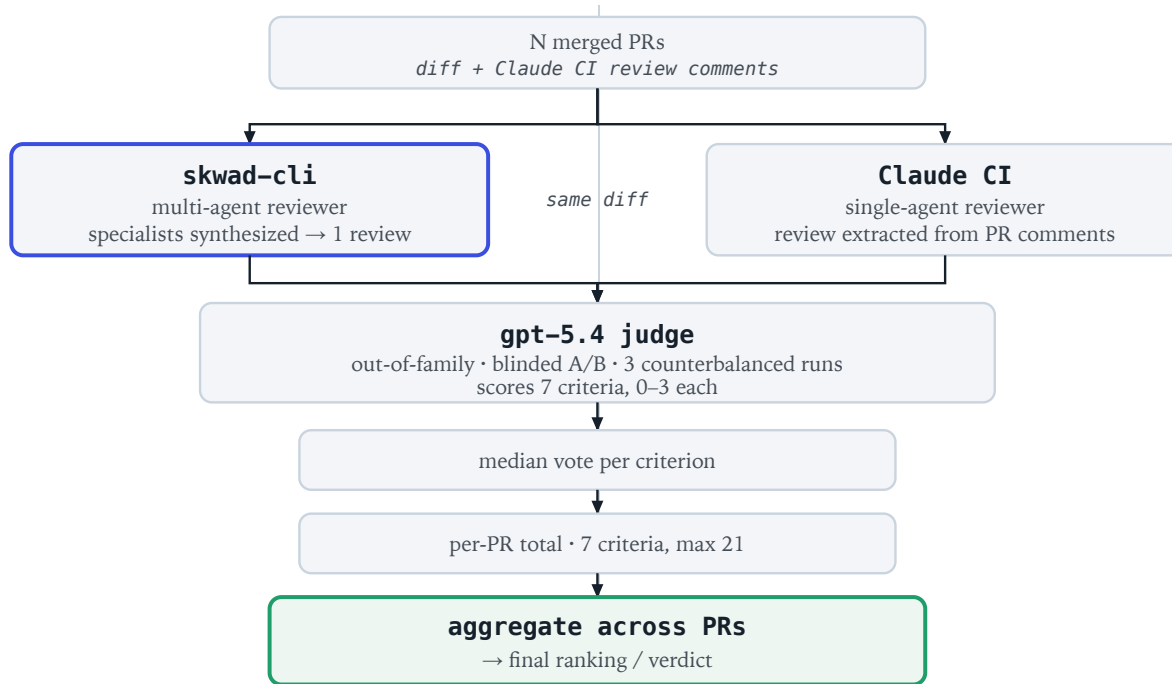


Figure 1. Evaluation overview. Both systems review the *identical* PR diff: skwad generates its review, while the single-agent Claude CI review is extracted from the PR’s existing comments (not regenerated). An out-of-family gpt-5.4 judge scores both under counterbalanced A/B blinding across three runs; per-criterion **median** votes roll up to a per-PR total (max 21) and aggregate across PRs to the final verdict.

2 Background

The LLM-as-judge paradigm [1] enables scalable evaluation but inherits the judge’s biases: position bias (favoring the first option), verbosity bias, and self-enhancement bias. Standard mitigations swap presentation order and use a judge from a different model family. We adopt both, and add a less common control: rather than trusting the judge’s reading, we require it to *verify each factual claim against the codebase*, separating substantive findings from confident but unsupported assertions [5,6].

3 Methodology

3.1 Hypotheses

H1: skwad reviews score higher than Claude CI reviews.
H0: no meaningful difference. The primary endpoint — total rubric score — and a two-sided Wilcoxon signed-rank test were *pre-registered* before results were inspected.

3.2 Systems under test

skwad spawns specialized reviewer agents on claude-sonnet-4-6 and synthesizes a single review. The baseline is GitHub’s Claude CI reviewer (claude-sonnet-4-5-20250929), a single-pass reviewer. Both received the *identical* PR diff at a pinned commit SHA.

3.3 Rubric

Seven criteria, each 0–3 (21 max): issue detection (ID), actionability (Act), severity accuracy (Sev), coverage (Cov),

signal-to-noise (S/N), depth (Dep), novel & substantive findings (Nov). The wide scale mitigates ceiling effects.

3.4 Judge & blinding

Scored by **gpt-5.4** (via codex) — a different family from both reviewers, so no in-family output can be favored. Each PR judged **three times** with counterbalanced slots (run 1: skwad=A; run 2: skwad=B; run 3: seeded random); per-criterion scores combined by **median vote** to suppress nondeterminism.

3.5 Grounding verification

For each PR the judge enumerated every factual claim in both reviews and checked it against the checked-out repository using read/grep tool calls (75–131 calls per PR), assigning *verified*, *contradicted*, *unverified*, or *non-falsifiable*. This yields a style-independent *verification rate*.

3.6 Statistical plan

Primary: Wilcoxon signed-rank on total score, Cliff’s δ , and a 95% bias-corrected-and-accelerated (BCa) bootstrap CI. Exploratory: per-criterion (7) and per-difficulty (3) Wilcoxon under Benjamini–Hochberg FDR [3] at $q=0.05$. Reliability: ordinal Krippendorff’s α [4], bootstrap 95% CI (nboot=200); criteria with an α lower bound < 0.60 are flagged as low-reliability and count against an overall harness-validity check — they are *not* dropped, and the headline statistics run on the full seven-criterion total.

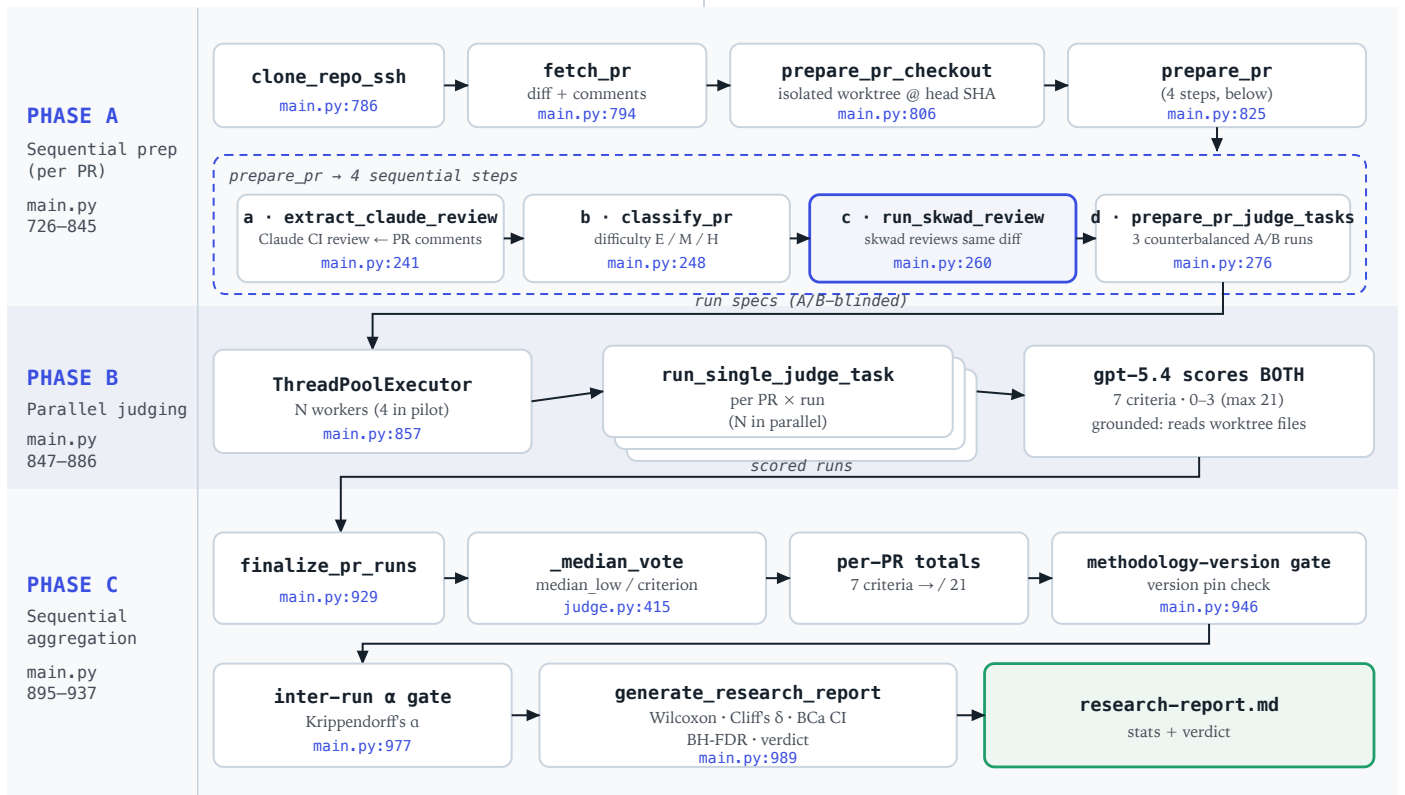


Figure 2. Detailed evaluation pipeline, faithful to the harness code. **Phase A** (sequential, per PR) clones the repo, fetches the diff and existing PR comments, checks out an isolated worktree at the PR head SHA, then runs `prepare_pr`'s four steps: extract the pre-existing Claude CI review from comments (a), classify difficulty (b), generate skwad's review of the *same* diff (c), and build three counterbalanced, A/B-blinded run specs (d). **Phase B** (parallel) fans the $\text{PR} \times \text{run}$ tasks across a worker pool (4 in the pilot, configurable); gpt-5.4 scores *both* reviews on the seven-criterion rubric (0–3, max 21), grounding every claim by reading files in the worktree. **Phase C** (sequential) collapses the three runs by per-criterion median vote, computes per-PR totals, applies the methodology-version gate and the Krippendorff's- α reliability gate, then emits the statistics and the report (`research-report.md`). Line anchors reference `main.py` / `judge.py` .

Repository	PRs	Difficulty
frontend-mos	2	hard, hard
watson	3	easy ×3
dash-api	3	hard, hard, med

Table 1. Sample: 8 PRs across 3 production repositories and 3 difficulty tiers; one further PR skipped (environment fault on resume).

Pull request	D	ID	Act	Sev	Cov	S/N	Dep	Nov	Total	W
dash-api#558	med	3-0	3-1	2-0	3-0	2-1	2-1	3-0	18-3	✓
dash-api#562	hard	3-2	3-2	1-0	3-1	2-1	3-1	3-1	18-8	✓
dash-api#565	hard	3-2	3-2	1-0	3-1	2-1	3-1	2-0	17-7	✓
frontend-mos#1816	hard	3-3	2-2	1-1	2-2	2-1	2-2	2-2	14-13	✓
frontend-mos#1818	hard	3-2	2-3	1-3	3-1	1-2	3-2	3-0	16-13	✓
watson#879	easy	3-1	3-2	1-1	3-1	1-2	2-1	2-0	15-8	✓
watson#890	easy	0-0	0-0	0-0	0-0	1-0	1-1	0-0	2-1	✓
watson#893	easy	3-2	3-2	1-1	3-1	2-1	2-1	3-0	17-8	✓

Table 2. Complete per-criterion median scores, `skwad`-CI (0–3 each). Ties shown grey; the few criteria where CI scored higher are marked in **amber** (Act/Sev/S-N on #1818; S/N on #879). `skwad` wins every *total*, but not every cell — evidence the aggregate win is broad-based, not a single-criterion artifact.

4 Results

4.1 Primary outcome

`skwad` scored strictly higher on all 8 PRs (8–0–0). The paired Wilcoxon test rejected H_0 at **$p=0.0078$** (statistic 0, $n=8$). Effect size was large: Cliff's **$\delta=0.78$** , 95% BCa CI [0.50, 1.00] — excluding zero, consistent with the test. Margins ranged from decisive (18–3 on dash-api#558) to razor-thin (14–13 on frontend-mos#1816).

4.2 Per-PR adjudication

Representative judge verdicts (skwad), with final totals:

dash-api#558 (med, 18–3) — Seven genuine issues confirmed: missing-key postmeta writes, an empty install fixture, missing PUT failure-path coverage, PUT non-atomicity, an unbounded `GetPosts`, client-visible internal terminology, and by-value `PartnerData` copying. CI's review carried 6 *contradicted* claims.

dash-api#562 (hard, 18–8) — Many code-borne issues: non-atomic multi-key writes, per-key upsert fanout, incomplete reset deletion, per-request regex compilation, cache-miss stampedes, and returning cached maps by reference.

dash-api#565 (hard, 17–7) — Seven verified issues: the non-agency `AgencyNetworkId` write path, `agency_network_ids=0` returning advertiser rows, missing `RowsAffected` handling, a `-1`-vs-`0` sentinel mismatch, missing agency scope in audit logs, tests not asserting the scope invariant, and silent advertiser fallback.

frontend-mos#1816 (hard, 14–13) — The tightest contest: five supported issues (over-broad non-S2S gate, empty validation status treated as eligible, skipped backend `e2e`, suppressed `PATCH` failure handling, dead `void` statements); two of skwad's findings were contradicted, and the systems tied on five of seven criteria.

frontend-mos#1818 (hard, 16–13) — 17 verified findings across correctness, performance, API design, observability, and tests. Notably CI *outscored* skwad on severity accuracy (3–1) and actionability (3–2) here — a genuinely split result that skwad won on coverage and novelty.

watson#879 (easy, 15–8) — Five supported issues: whitespace-only tokens passing the guard, a new test omitting any `install_time` assertion, an older test checking only `install_id`, an unused fixture field, and a hardcoded value.

watson#890 (easy, 2–1) — The pilot's low-water mark: a trivial PR adding a `TraceID` field. Both reviews found almost nothing, and this is the *only* PR where skwad's verification rate (24%) fell below CI's (40%) — the judge faulted skwad for missing the `protoutil` conversion gap. A point worth stating plainly rather than burying.

watson#893 (easy, 17–8) — A clear cache-hydration behavioral bug plus secondary issues: missing tests, zero-value emission, token logging, and type consistency.

4.3 Grounding: substance over style

Across 643 adjudicated claims, skwad's claims verified against code at **70.9%** vs **60.2%** for the baseline (Table 3). The baseline also produced more *contradicted* claims (37 vs 30) — assertions the repository disproves. Because verification is computed from code, not prose, it directly rebuts the "skwad just writes more" concern. The lone exception (watson#890) is disclosed above.

Pull request	skwad – ver / grnd / contra		Claude CI – ver / grnd / contra			
dash-api#558	76%	94%	0	56%	96%	6
dash-api#562	72%	97%	5	45%	90%	7
dash-api#565	81%	96%	5	65%	93%	6
frontend-mos#1816	72%	100%	7	59%	96%	6
frontend-mos#1818	86%	85%	4	81%	100%	2
watson#879	71%	91%	4	61%	84%	0
watson#890	24%	80%	2	40%	91%	7
watson#893	81%	92%	3	71%	92%	3
Mean	70.9%	91%	30	60.2%	93%	37

Table 3. Per-PR claim verification rate, reasoning-grounding rate, and count of *contradicted* claims. skwad leads on verification in 7 of 8 PRs (watson#890 the exception, in **amber**) and produces fewer contradicted claims overall.

4.4 Inter-rater reliability

Ordinal Krippendorff's α (Table 4) shows the judge scored objective criteria highly consistently; three subjective criteria fell below the 0.60 gate and are flagged as a reliability caveat (and trip the harness-validity check), but they are *not* excluded — the Wilcoxon, Cliff's δ , and verdict all run on the full seven-criterion, 21-point total.

Criterion	α	95% CI	Gate
Coverage	0.97	[.91, 1.0]	pass
Novel findings	0.93	[.83, .99]	pass
Issue detection	0.92	[.73, .99]	pass
Depth	0.86	[.64, .97]	pass
Actionability	0.68	[.31, .89]	below
Signal-to-noise	0.57	[.16, .84]	below
Severity accuracy	0.50	[.05, .81]	below

Table 4. Gate = α lower-CI bound $\geq$ 0.60 (bootstrap, nboot=200).

4.5 Exploratory tests & bias checks

After Benjamini-Hochberg correction (Table 5) *no* individual criterion or difficulty test reached significance (smallest adjusted $p=0.070$): at $N=8$ the breakdown is underpowered, so we make no per-criterion claims. The position-bias check was clean for skwad (run-1-A vs run-2-B, $p=0.53$) — the win is not a presentation-order artifact — while the baseline's was weaker ($p=0.094$).

Test	raw p	BH p	sig
Issue detection	.031	.070	–
Coverage	.031	.070	–
Depth	.031	.070	–
Novel findings	.031	.070	–
Hard (difficulty)	.125	.225	–
Actionability	.188	.281	–
Easy (difficulty)	.250	.321	–
Signal-to-noise	.289	.325	–
Severity accuracy	.750	.750	–

Table 5. Exploratory Wilcoxon tests, BH-FDR at $q=0.05$. None survive correction at $N=8$ — reported in full to avoid the appearance of significance the data do not support.

5 Threats to Validity

- **Judge stylistic bias (primary residual).** A GPT-family judge may reward structure and finding-count, plausibly inflating the multi-agent margin. Mitigated by the style-immune verification rate and by gating out low-reliability subjective criteria.
- **Sample selection.** PRs were curated from our repositories; the confirmatory run should sample more broadly and blindly.
- **Statistical power.** At $N=8$ only large effects are reliably detectable; a planned $N=30$ run resolves medium effects ($\delta \approx 0.33$) and the per-criterion picture.
- **Nondeterminism.** Mitigated by 3-run median voting.
- **Prompt sensitivity.** A single rubric/persona version was used, SHA-pinned for reproducibility.

6 Discussion

The decisive aggregate win (Table 2) is broad-based: skwad leads on most criteria of most PRs, yet the baseline still wins individual cells (severity and actionability on #1818, S/N on #879) and even a verification round (#890). This texture is reassuring — a uniform sweep would suggest a judge artifact, whereas a broad-but-imperfect lead is what a genuinely stronger reviewer produces. The grounding result is the load-bearing finding: skwad's edge survives the one metric that prose cannot inflate.

7 Conclusion

Under a cross-model judge and per-claim grounding, a multi-agent reviewer decisively outperformed a strong single-agent baseline — winning every comparison with a large, significant effect, on *verifiable* findings rather than stylistic volume. The evidence supports H1 as a pilot. We recommend adopting skwad as the CI reviewer via a low-risk shadow rollout while a pre-registered 30-PR confirmatory study locks the magnitude and per-criterion profile.

References

1. Zheng, L. et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS D&B, 2023.
2. Saito, K. et al. *Verbosity Bias in Preference Labeling by LLMs*. 2023.
3. Benjamini, Y. & Hochberg, Y. *Controlling the False Discovery Rate*. J. R. Statist. Soc. B 57(1):289–300, 1995.
4. Krippendorff, K. *Computing Krippendorff's Alpha-Reliability*. Univ. of Pennsylvania, 2011.
5. Romano, J. et al. *Evaluating group differences: Cliff's δ* . 2006.
6. Efron, B. & Tibshirani, R. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

skwad-cli empirical evaluation · pilot N=8 · judge gpt-5.4 (cross-model) · primary endpoint pre-registered · 643
claims adjudicated · companion research report contains full claim traces, raw reviews, and manifest